

CS339: Abstractions and Paradigms for Programming

Object Layouts

Manas Thakur
CSE, IIT Bombay



Autumn 2026

Recall from the last class

In Java, fields are bound statically, while methods dynamically.

- We saw this with the `Animal/Monkey` example:
 - Fields were accessed based on the static type, whereas methods were accessed based on the dynamic (the actual object's) type.
 - We were able to access a shadowed field from the parent, but a call to an overridden method always dispatched to the child's implementation.

Today: we see how is this made possible, simply by understanding how objects are laid out in the memory and how the language is lowered down.



Assumptions

- C-style semantics: no classes, no objects, no method calls
- Memory allocated (using `alloc`) is laid out as a 1D array, which can be accessed through pointers
- Every object, every field, every method is just “some bytes at some offset”
- The memory is addressed in units of 4 bytes



Running Example: class A

```
class A {  
    int x;  
    int y;  
    int m1(int z) {  
        int t1 = x + y + z;  
        return t1;  
    }  
    int m2() {  
        return x;  
    }  
    void m3() {  
        // some code  
    }  
    void m4() {  
        // some code  
    }  
}
```



```
class A {  
    int x;  
    int y;  
    int m1(int z) {  
        int t1 = this.x+this.y+z;  
        return t1;  
    }  
    int m2() {  
        return this.x;  
    }  
    void m3() {  
        // some code  
    }  
    void m4() {  
        // some code  
    }  
}
```

Methods to plain functions

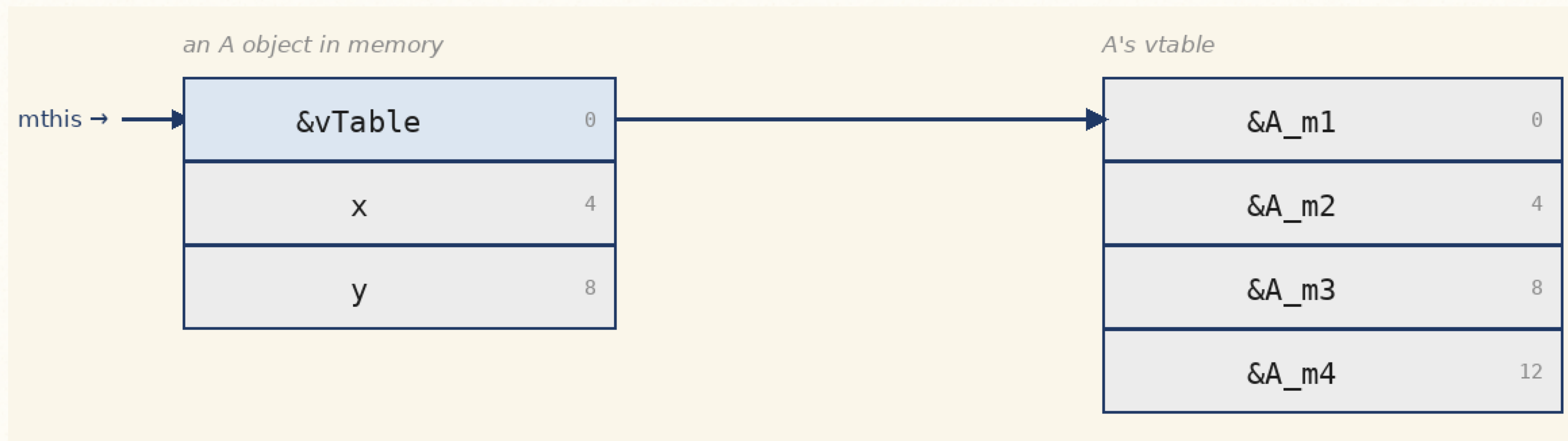
```
class A {  
    int x;  
    int y;  
    int m1(int z) {  
        int t1 = this.x+this.y+z;  
        return t1;  
    }  
    int m2() {  
        return this.x;  
    }  
    void m3() {  
        // some code  
    }  
    void m4() {  
        // some code  
    }  
}
```



```
int A_m1(A mthis, int z) {  
    int t1 = mthis.x+mthis.y+z;  
    return t1;  
}  
  
int A_m2(A mthis) {  
    return mthis.x;  
}  
  
...
```

Layout of an A instance

```
class A {  
    int x;  
    int y;  
    int m1(int z) {...}  
    int m2() {...}  
    void m3() {...}  
    void m4() {...}  
}
```



Lowering constructors and method calls

```
A a = new A();
```

```
x1 = alloc(12);
```

```
x1[4] = 0;
```

```
x1[8] = 0;
```

```
x2 = alloc(16);
```

```
x2[0] = &A_m1;
```

```
x2[4] = &A_m2;
```

```
x2[8] = &A_m3;
```

```
x2[12] = &A_m4;
```

```
x1[0] = x2;
```

```
a = x1;
```

```
int r = a.m1(5);
```

```
t1 = a[0];
```

```
int r = t1[0](a, 5);
```

```
a.m4();
```

```
t1 = a[0];  
t1[12](a);
```

Field accesses

```
int A_m1(A mthis, int z) {  
    int t1 = mthis.x+mthis.y+z;  
    return t1;  
}
```



```
int A_m1(A mthis, int z) {  
    int t1 = mthis[4]  
            + mthis[8]  
            + z;  
    return t1;  
}
```

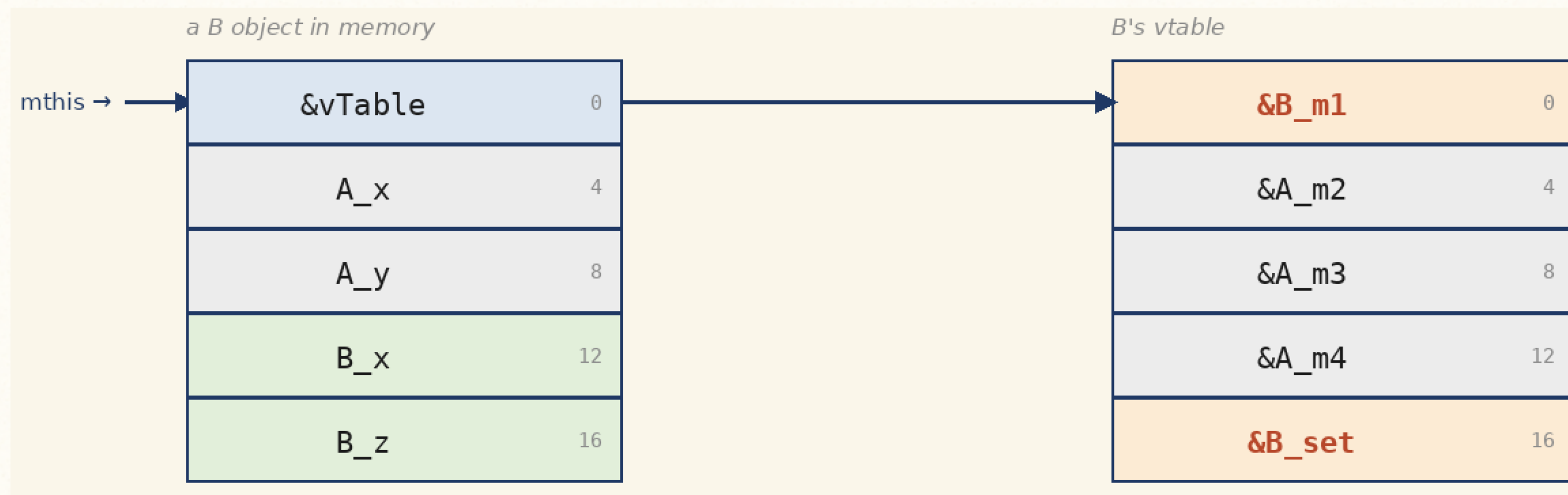
Running Example: classes with inheritance

```
class A {  
    int x;  
    int y;  
    int m1(int z) {  
        int t1 = x + y + z;  
        return t1;  
    }  
    int m2() {  
        return x;  
    }  
    void m3() {  
        // some code  
    }  
    void m4() {  
        // some code  
    }  
}
```

```
class B extends A {  
    int x;  
    int z;  
    int m1(int p) {  
        int t2 = m2();  
        int t3 = 2*x + y - p + t2;  
        return t3;  
    }  
    void set(int a, int b) {  
        x = a;  
        z = b;  
    }  
}
```

Layout of a B instance

```
class A {  
    int x;  
    int y;  
    int m1(int z) {...}  
    int m2() {...}  
    void m3() {...}  
    void m4() {...}  
}  
class B extends A {  
    int x;  
    int z;  
    int m1(int p) {...}  
    void set(int a, int b) {...}  
}
```



Field accesses in class B

```
int m1(int p) {  
    int t2 = m2();  
    int t3 = 2*x + y - p + t2;  
    return t3;  
}
```



```
int B_m1(B mthis, int p) {  
    int t2 = mthis[0][4](mthis);  
    int t3 = 2*mthis[12]  
            + mthis[8]  
            - p + t2;  
    return t3;  
}
```

Which field is accessed in the A_m2 call?

```
class A {  
    int x;  
    int y;  
    ...  
    int m2() {  
        return this.x;  
    }  
    ...  
}
```

```
int B_m1(B mthis, int p) {  
    int t2 = mthis[0][4](mthis);  
    int t3 = 2*mthis[12]  
            + mthis[8]  
            - p + t2;  
    return t3;  
}
```

Each Java class is compiled separately:

```
int m2(A mthis) {  
    return mthis[4];  
}
```

**Fields are bound statically,
methods dynamically.**



Unevaluated...



Next class: writing one function that works for every type — without writing it more than once.